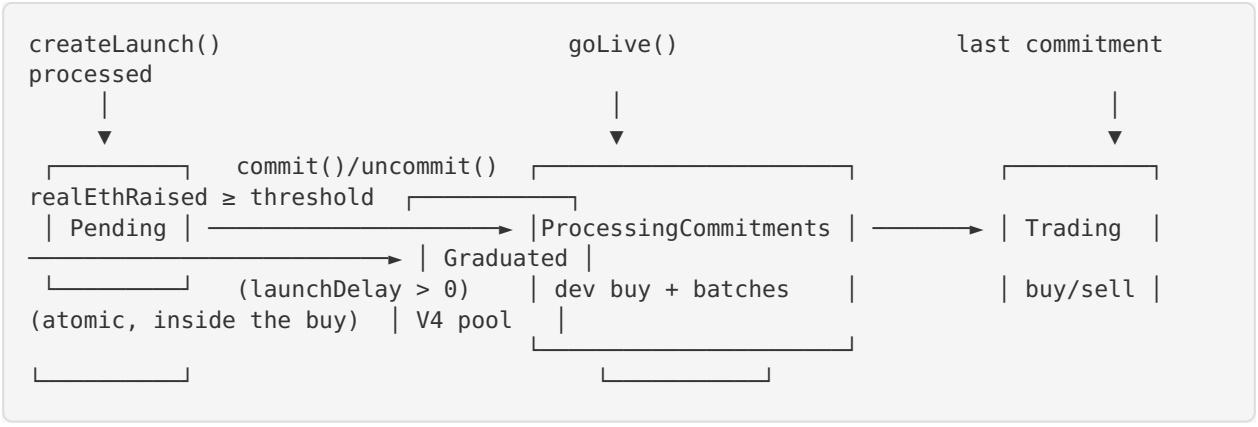
Polydash Launchpad — Architecture

Target chain: **Robinhood Chain** (Arbitrum Orbit L2, chain id `4663`).
Default deployment target for this repo: **Arbitrum Sepolia** (`421614`). The mainnet deploy path is hard-blocked
(see `script/Deploy.s.sol`) until there is a completed external audit **and** written approval.

1. Contracts

Contract	Role	Deployed
`PolydashFactory`	Creates launches, holds global config for **future** launches, routes platform fees. `Ownable2Step` (owner should be a multisig).	once
`PolydashLaunch`	Per-token bonding curve: commitments → dev buy → trading → graduation. Holds curve ETH, creator fees, platform fees, refunds.	EIP-1167 clone per launch
`PolydashToken`	Fixed-supply ERC-20 (burnable). Whole supply minted once to the launch. No owner, mint, pause, blacklist or tax.	`new` per launch
`PolydashHook`	Uniswap V4 hook. Creates the graduated ETH/token pool, holds the full-range LP **forever**, charges platform + creator fees in ETH on every swap.	once (CREATE2-mined address)
`BuybackRouter`	Optional. Swaps a configurable share of platform fees into $POLYDASH on Uniswap V3 (TWAP-protected) and sends it to the Trading Rewards pool. Disabled by default.	optional
`libraries/FeeMath`	Single source of truth for fee splits (platform fee constant = 20 bps).	library
`libraries/BondingCurveMath`	Constant-product virtual-AMM math, 512-bit `mulDiv`, rounding in favour of the pool.	library

2. Launch lifecycle



- **Pending** – only if `launchDelay > 0`. Users `commit()` ETH and may `uncommit()` it in full.
- **goLive()** – permissionless once `launchTime` is reached (the factory calls it immediately when `launchDelay == 0`).
  The creator's dev buy executes first, then up to 200 commitments are filled at curve prices (normal platform + creator fee, no anti-sniper surcharge, no wallet cap). Remaining batches via `processCommitmentBatch()` (permissionless).
- **Trading** – `buy(minOut)` / `sell(tokensIn, minEthOut)` against the curve.
- **Graduation** – the buy that reaches the threshold is capped exactly at the threshold, unused ETH is returned
  in the same transaction, and `_graduate()` runs atomically at the **end** of that `buy` / `goLive` / `processCommitmentBatch` call (state and event first, external calls last). Commitments processed after graduation are refunded
  through the pull-based `claimRefund()`. `processCommitmentBatch()` stays callable after graduation until every
  committer is processed, and any committer can also pull an unprocessed commitment directly with
  `withdrawUnprocessedCommitment()` — no commitment can get stuck.
- **PolydashCommitVault** (option-
  al) – pools users' commitments into one launch commitment; after `settle(launch)`
  every depositor claims the same average-price share of tokens plus refunds. No owner/fee/admin.

3. Bonding curve math

Constant product on virtual reserves:

```
ethReserve   = virtualEth + realEthRaised          (default virtualEth = 1.5 ETH)
tokenReserve = curveSupply - tokensSold            (curveSupply = 80% of totalSupply)
k            = virtualEth * curveSupply            (fixed at launch)

tokensOut(netEthIn) = tokenReserve - ceil(ethReserve * tokenReserve / (ethReserve +
netEthIn))
grossEthOut(tokIn)  = ethReserve   - ceil(ethReserve * tokenReserve / (tokenReserve +
tokIn))
spotPrice           = ethReserve * 1e18 / tokenReserve
```

- New reserves are always rounded **up**, so `ethReserve * tokenReserve` never decreases (fuzz + in-variant tested).

- 20% of supply is held back as the LP reserve.
- **Graduation threshold** – owner picks one mode for future launches:
- `EthRaised` (default 15 ETH real ETH raised), or
- `MarketCap` – the ETH reserve where `spotPrice * totalSupply == target`:
  `E = sqrt(mcap * k / totalSupply)`, threshold `= E - virtualEth`.
  The resolved threshold is frozen into each launch at creation.

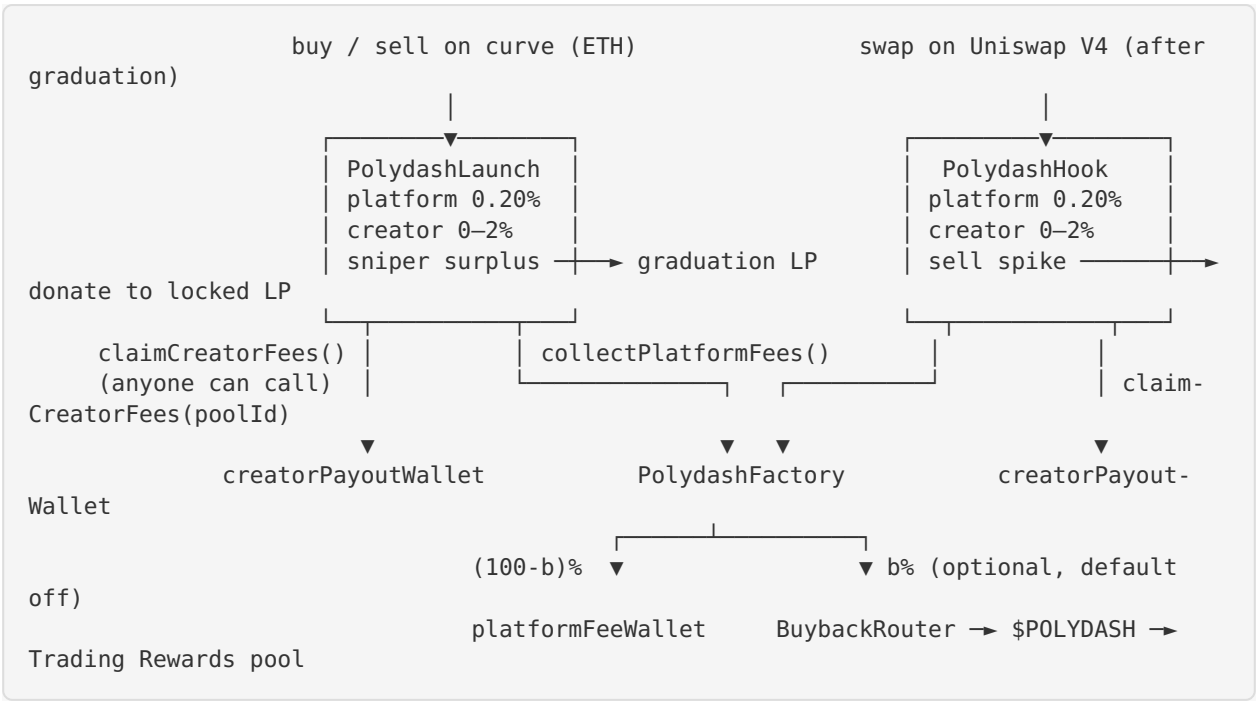
4. Fees

All fees are taken **in ETH**, computed on the gross ETH amount, each component floored independently (`FeeMath.split`):

Fee	Rate	Where it goes
Platform	**20 bps, hard-coded** (no setter anywhere)	`platformFeesAccrued` → factory → platform fee wallet (minus optional buyback share)
Creator	0–200 bps, chosen at launch, immutable	`creatorFeesAccrued` → only ever `creatorPayoutWallet`
Anti-sniper surplus	decays linearly from `antiSniperFeeBps − creatorFeeBps` to 0 over `antiSniperFeeDuration` (public buys only)	**nobody** — added to the permanently locked graduation liquidity
Post-graduation sell spike (optional, ≤10%, ≤1h)	sells only, decays to 0 at `sellSpikeEnd`	donated to the locked V4 pool liquidity

Anti-sniper wallet cap: for the first `antiSniperCapBlocks` blocks of trading, a wallet can hold at most `antiSniperCapBps` of total supply bought from the curve.

Fee flow



5. Graduation to Uniswap V4

`PolydashLaunch._graduate()`:

1. LP ETH = `realEthRaised + sniperTaxAccrued`.
2. LP tokens are sized at the **final curve spot price** (`lpEth * tokenReserve / ethReserve`) so the V4 pool opens
   at exactly the curve's last price — no discontinuity to arbitrage.
3. All remaining unsold tokens beyond that are **burned**.
4. `PolydashHook.deployGraduatedPool()` initialises an ETH/token pool (`fee = 0`, `tickSpacing = 200`, `hooks = PolydashHook`) and adds a full-range position owned by the hook. Rounding dust: tokens are burned,
   ETH dust is sent to `0x…dEaD`.

LP is locked forever: the hook contains no code path that calls `modifyLiquidity` with a negative delta, the
unlock callback only works while `_deploying` is set during graduation, and `beforeInitialize` always reverts so no
one else can create pools that use this hook.

Hook swap fee mechanics (all in ETH, never in the launched token):

Swap type	ETH is…	Charged in
exact-in buy	specified	`beforeSwap` (fee taken out of ETH paid)
exact-out sell	specified	`beforeSwap` (pool releases amount + fee)
exact-in sell	unspecified	`afterSwap` (fee deducted from ETH out)
exact-out buy	unspecified	`afterSwap` (fee added on top)

Why V4 (and not V3) for graduated pools — trade-offs

	Uniswap V4 + hook (chosen)	Uniswap V3 fallback
Creator fee after graduation	Enforced on-chain on every swap by the hook	Not possible without a taxed token (breaks composability)
Fee currency	Always ETH	LP fees accrue in both tokens
LP lock	Structural — hook owns the position and has no removal code	Need to send the NFT to a locker / dead address
Pool spam / fake pools	`beforeInitialize` reverts for third parties	Anyone can create other fee-tier pools
Aggregator / router support	Newer; depends on V4 routing on Robinhood Chain	Mature
Gas	Hook adds ~a few tens of k gas per swap	Cheaper swaps

A V3 fallback was deliberately **not** implemented: it would silently drop the post-graduation creator fee, which is a
product promise. V3 is used only by the optional `BuybackRouter` (where the $POLYDASH liquidity currently lives).

6. Per-launch clones

Each launch is an EIP-1167 minimal proxy of a single `PolydashLaunch` implementation:

- Cheap creation (~1.1M gas average for `createLaunch`, including deploying the token).
- `initialize()` can run exactly once (`phase` guard) and only sets the factory to `msg.sender`.
- The implementation is **not upgradeable** and the factory's `launchImplementation` is `immutable`.

- Each launch is isolated: its ETH, fees and refunds are held in its own contract.
- Config changes by the factory owner only affect launches created **after** the change.

7. Security properties (tested)

- Platform fee is exactly `floor(gross * 20 / 10_000)` — fuzzed.
- `k` never decreases; launch ETH balance always ≥ accrued fees + refunds + `realEthRaised` (invariant suite).
- All state-changing entry points use `nonReentrant`; malicious payout wallets / re-entrant buyers tested.
- Creator fees can only reach `creatorPayoutWallet` (see `CREATOR_FEE_SAFETY.md`).

8. Future work (not in this release)

- **Staking V2 integration** — route a share of platform fees / buybacks into Polydash Staking V2 (factory `0x904b8F5d95B593b5F4a6Dbb116b58e1d5CBe93f0` on Robinhood Chain; recorded as a constant, unused on-chain).
- **dasheco.xyz** — ecosystem front-end / indexer integration (launch discovery, creator dashboards).
- **One-click staking** — let creators spin up a staking pool for their graduated token from the launch page.
- Optional V3 graduation mode, only if the creator explicitly opts out of post-graduation creator fees.
- External audit, then mainnet deployment (requires written approval).